



1장 : 머신 러닝과 딥러닝

1.1 인공지능, 머신 러닝과 딥러닝

인공지능 (Artificial Intelligence, AI)

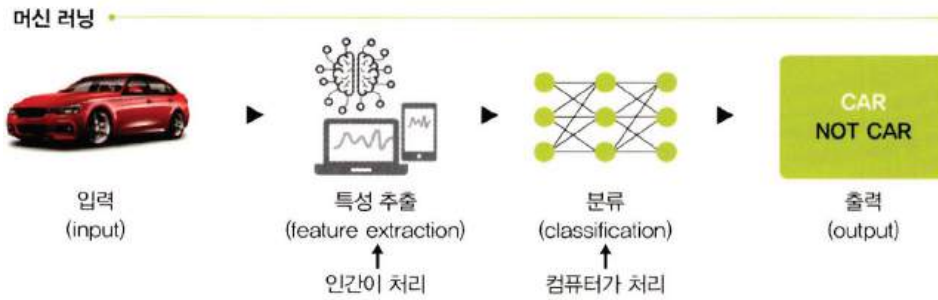
- 사람이 하는 일을 컴퓨터가 할 수 있도록 하는 기술
- 구현 방법
 - 머신 러닝 (machine learning)
 - 딥러닝(deep learning)
- AI, ML, DL 의 관계



머신 러닝과 딥러닝 차이

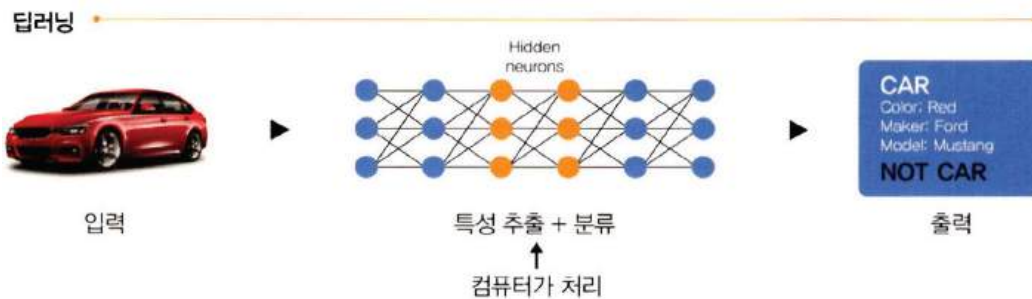
머신 러닝

- 주어진 데이터를 컴퓨터가 인식 가능한 형태로 사람이 전처리 해야함
 - 데이터의 특징을 추출 해줘야함



딥러닝

- 사람이 하던 처리 작업 생략
 - 데이터를 신경망에 적용 → 컴퓨터가 스스로 찾음



비교 차트

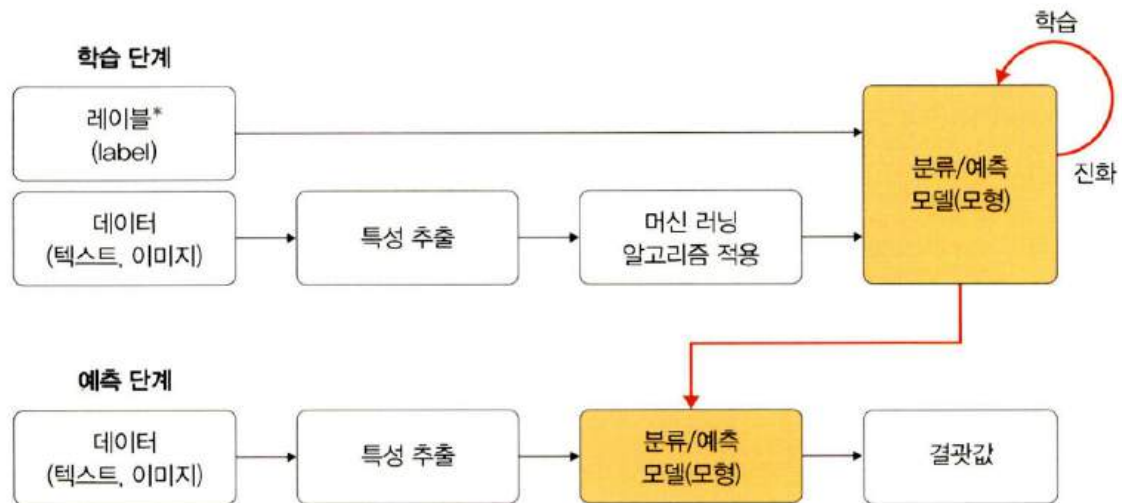
구분	머신 러닝	딥 러닝
동적 원리	입력한 데이터에 algorithms 적용하며 예측함	신경망을 통해 데이터 특징 및 관계 파악
재사용	같은 algorithm을 다른 유형의 데이터에 바로 재사용 하기 어려움 (예: text data 분석용으로 만든 decision tree를 이미지 데이터에 쓰기 힘들)	신경망 구조를 다양한 데이터 유형에 재사용 가능(예: CNN은 원래 이미지 데이터에 사용하지만 텍스트 나 음성 데이터에서도 적용 가능)
데이터	수천 개면 충분할 수 있음	수백만 개 이상의 데이터 필요
훈련 시간	단시간	장시간
결과	점수 또는 분류 등 숫자 값 (예: 점수-0.87% probability of spam, "Spam" or "Not Spam")	점수, 텍스트, 소리 다 가능 (예: Text generation → ChatGPT, Image generation → DALL.E)

1.2 머신 러닝이란

- 컴퓨터가 대용량 데이터에서 지식이나 패턴을 찾아 학습하고 예측할 수 있게 하는 알고리즘과 기술을 개발하는 분야

1.2.1 머신 러닝 학습 과정

- 크게 학습 단계(learning)와 예측 단계(prediction)로 구분



예: Spam Detection

- 학습 단계

1. 데이터 수집

- Two parts
 - 텍스트 데이터 : "you won free money! Click here"
 - 레이블 : Spam

2. 특성 추출 (Feature extraction)

- 특정 단어의 빈도 (예: "free", "win", "money", "click")
- 발신자 주소 (신뢰할 수 있는 도메인 vs. 알 수 없는 도메인)
- "you won free money! Click here "https://xxx" and here "http://xxx"

"free" 포함	"click" 포함	링크 개수	레이블
1	1	2	Spam

- 대표 적인 방법들

- General : PCA, LDA, Autoencoder
 - Text Data : BoW, TF-IDF, Word2Vec, BERT
 - Image Data : raw pixels, HOG, CNN
 - Audio : MFCC
3. 머신러닝 알고리즘 적용
 - Logistic Regression
 4. 모델 학습
 - 이런 정보들을 알게 됨
 - "free", "click", "win" → 스팸 신호 강함
- 예측 단계
 1. 새로운 데이터
 - 새로운 메일 : "click here to win a free iPhone 'https://xxx' "
 2. 특성 추출
 - "free" 포함 → 1
 - "click" 포함 → 1
 - "win" 포함 → 1
 - 링크 개수 = 1
 3. 모델 예측/분류
 - $P(\text{Spam})=0.98$
 4. 결과 제공
 - "Spam"

머신 러닝의 주요 구성 요소

- 데이터
 - 실제 데이터의 특징이 잘 반영되고 편향 되지 않는 데이터 확보가 중요
 - 학습 데이터를 대부분

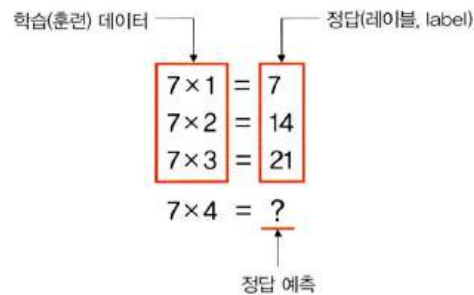
3. 평가를 바탕으로 모델 업데이트

- 3 단계를 반복함 → 최고의 모델 찾음

1.2.2 머신 러닝 학습 알고리즘

1. 지도 학습 (Supervised Learning)

- 정답이 있는 데이터로 학습
- 데이터 형태 → 입력 데이터 & 정답 쌍



- 예: Spam Mail 분류

비지도 학습 (Unsupervised Learning)

- 정답이 없는 데이터 속 숨겨진 패턴이나 군집 찾기



- 지도 학습은 A, B 이렇게 정확한 결과가 있지만 비지도 학습은 없음(비슷한 데이터끼리 모아 놓은 클러스터만 존재)
- 데이터 형태 → 입력만 있음
- 예: 고객 세분화

- 고객들의 구매 이력(금액, 빈도) → 알고리즘으로 유사한 행동을 하는 고객들을 묶음 (많이 사는 고객, 세일 때만 사는 고객) → 활용 (각 그룹별 맞춤형 광고)

강화학습 (Reinforcement Learning)

- 액션을 힘 → 보상을 받음 → 학습됨
 - 에이전트(Agent)가 환경(Environment) 속에서 행동(Action)을 하며 보상(Reward) 을 최대화하도록 학습



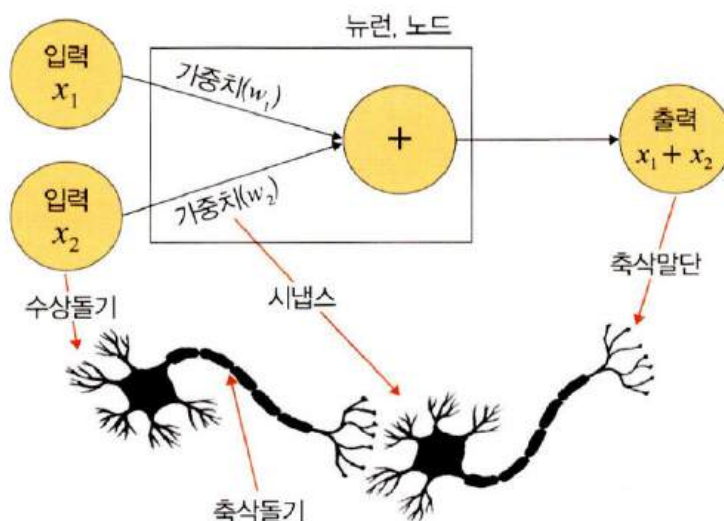
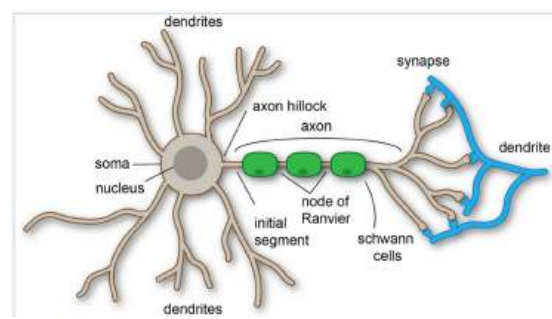
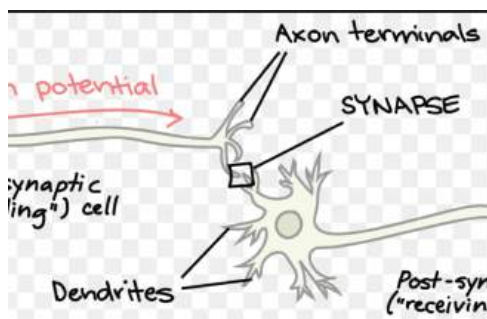
- 예 : AlphaGo

자주 사용되는 알고리즘

구분	유형	알고리즘
지도 학습 (supervised learning)	분류(classification)	• K-최근접 이웃(K-Nearest Neighbor, KNN) • 서포트 벡터 머신(Support Vector Machine, SVM) • 결정 트리(decision tree) • 로지스틱 회귀(logistic regression)
	회귀(regression)	선형 회귀(linear regression)
비지도 학습 (unsupervised learning)	군집(clustering)	• K-평균 군집화(K-means clustering) • 밀도 기반 군집 분석(DBSCAN)
	차원 축소 (dimensionality reduction)	주성분 분석 (Principal Component Analysis, PCA)
강화 학습 (reinforcement learning)	—	마르코프 결정 과정 (Markov Decision Process, MDP)

1.3 딥러닝이란

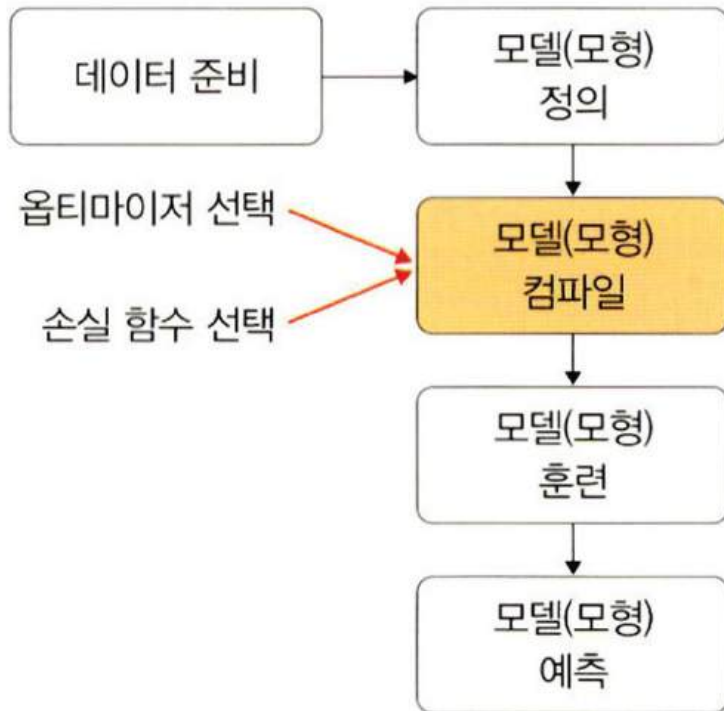
- 인간의 신경망 원리를 모방한 심층 신경망 이론을 기반으로 고안된 머신 러닝 방법의 일종
 - 뇌는 수많은 뉴런(neuron) 과 시냅스(synapse)로 구성
 - 각 뉴런은 복잡하게 연결되어 병렬 연산을 수행함
 - 덕분에 기존 컴퓨터가 처리하기 어려웠던 음성 인식, 영상 인식 같은 복잡한 작업이 가능해졌음
- 인간의 신경망 원리를 모방한 심층 신경망



- 두개의 비교
 - 생물학적 뉴런의 부분 (수상돌기, 축삭돌기, 시냅스, 축삭말단)
 - 인공 뉴런 모델 (입력, 가중치, 가중합, 출력)
- 흐름:

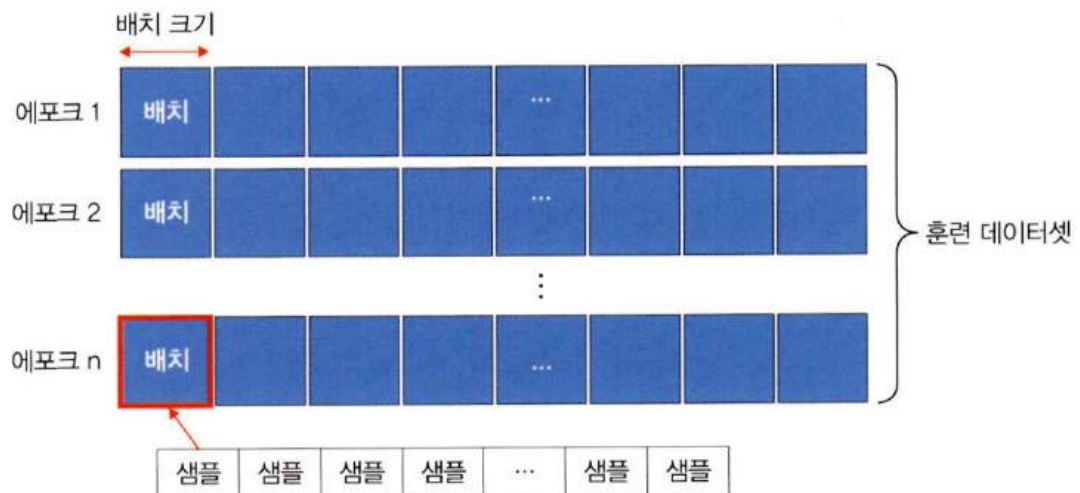
- 입력값 (x_1, x_2) 이 들어옴.
- 각 입력에는 중요도를 조절하는 가중치 (w_1, w_2) 가 곱해짐
- 가중치가 곱해진 입력값을 모두 합산함
- 그 결과를 활성화 함수(activation function)에 통과시켜 출력을 생성
- 생물학적 뉴런의 구성 요소
 - 수상돌기 (Dendrite):
 - 주변 뉴런에서 신호를 받아들임.
 - 이 신호를 전기적 신호로 변환해 세포체와 축삭돌기로 보냄
 - 시냅스 (Synapse):
 - 뉴런과 뉴런을 연결하는 부위.
 - 한 뉴런의 축삭돌기와 다른 뉴런의 수상돌기가 만나는 지점
 - 축삭돌기 (Axon):
 - 다른 뉴런으로 신호를 전달하는 긴 섬유 구조
 - 축삭말단 (Axon Terminal):
 - 축삭돌기의 끝부분
- 생물학적 뉴런:
 - 수상돌기가 신호를 받음 → 세포체에서 처리 → 축삭돌기가 신호 전달 → 축삭말단이 시냅스를 통해 다음 뉴런으로 전달
- 인공 뉴런 (딥러닝):
 - 입력값과 가중치 → 가중합 → 활성화 함수 → 출력 → 다음 층으로 전달

1.3.1 딥러닝 학습 과정



- 데이터 준비
 - 이미 전처리 한 데이터
 - 파이토치 → <https://tutorials.pytorch.kr/>
 - 카라스 → <https://keras.io/>
 - 캐글 (Kaggle)
- 모델(모형) 정의
 - 신경망을 생성하는 단계
 - 은닉층(hidden layer) 개수가 많을수록 성능이 좋아지지만 과적합이 발생 할 수 있음
 - 모델 정의 단계에서 신경망을 제대로 생성하는게 중요함
- 모델(모형) 컴파일
 - 활성화 함수(activation function), 손실 함수(loss function), 옵티마이저(optimizer)를 선택하는 단계
 - activation function : 입력 신호가 일정 기준 이상 → 출력 신호로 변환
 - 시그모이드, 하이퍼볼릭 텐젠트, 렐루
 - loss function : 모델 출력 값과 레이블의 값 차이, 오차를 구하는 함수

- 평균 제곱 오차(mean squared error)
- cross entropy error
- optimizer : loss function기반으로 네트워크 업데이트 방법 결정, 업데이트 결정 방법에 사용되는 것 : Adam, RMSProp
- 데이터 형태의 따라 다양한 option이 가능
 - 연속형 : 평균 제곱 오차(mean squared error, MSE)
 - 이진 분류(binary classification) : cross entropy
- 모델(모형) 훈련
 - 한 번에 처리할 데이터양 지정
 - 많으면 → 속도 느림, 메모리 부족 문제
 - 훈련 과정에서 값의 변화를 시각적으로 확인하며 파라미터와 하이퍼파라미터에 대한 최적 값을 찾을 수 있어야함
 - 훈련 데이터셋 = 학습 데이터셋
 - 전체 훈련 데이터셋에서 → 일정한 묶음으로 처리 할 수 있는 배치와 훈련의 횟수 epoch 선택이 중요함

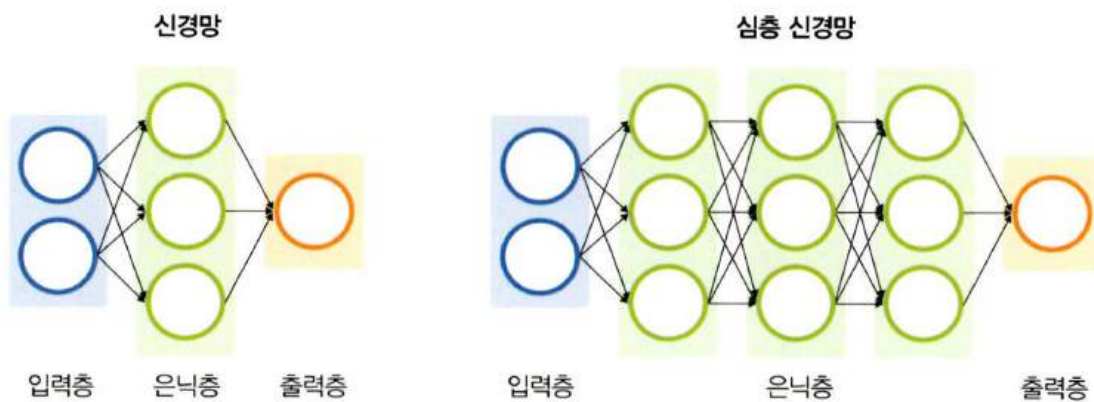


- 훈련 데이터셋 = 1000개
 - 배치 크기 : 20 → 샘플 단위 20개마다 가중치 업데이트 → 총 50번 (1000/20)의 가중치가 업데이트됨
 - epoch : 10 → 가중치를 50번 업데이트하는 것을 총 10번 반복 → 총 500번 업데이트됨

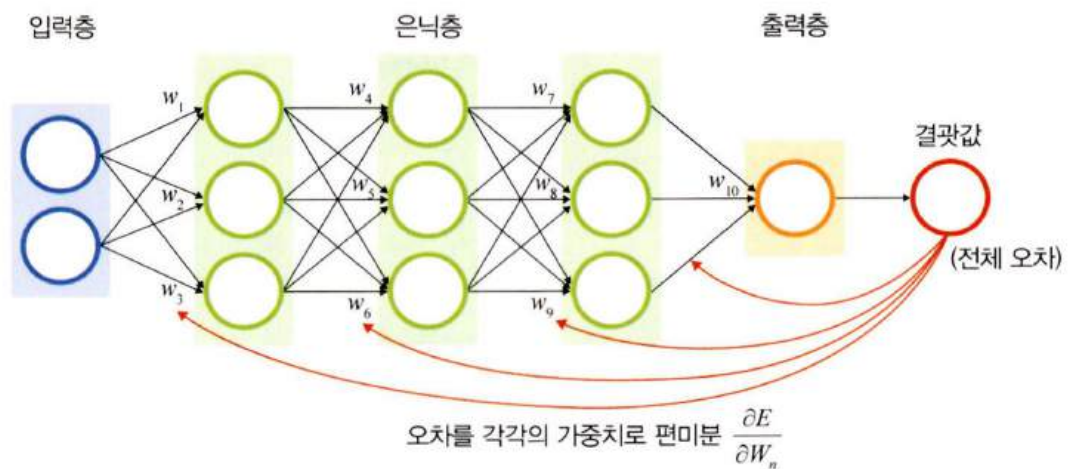
- 모델(모형) 예측
 - 검증 데이터셋을 생성한 모델에 적용 → 예측 진행하는 단계
 - 예측력이 낮다면 파라미터 튜닝하거나, 신경망 자체를 재설계해야함

신경망과 역전파

- 심층 신경망(deep neural network)
 - 은닉층이 두 개 이상
 - 데이터셋의 어떤 특성들이 중요한지 스스로 가르쳐 줄 수 있는 기능



- 역전파(backpropagation)
 - 가중치 값 업데이트
 - 역전파 계산 : 미분 사용

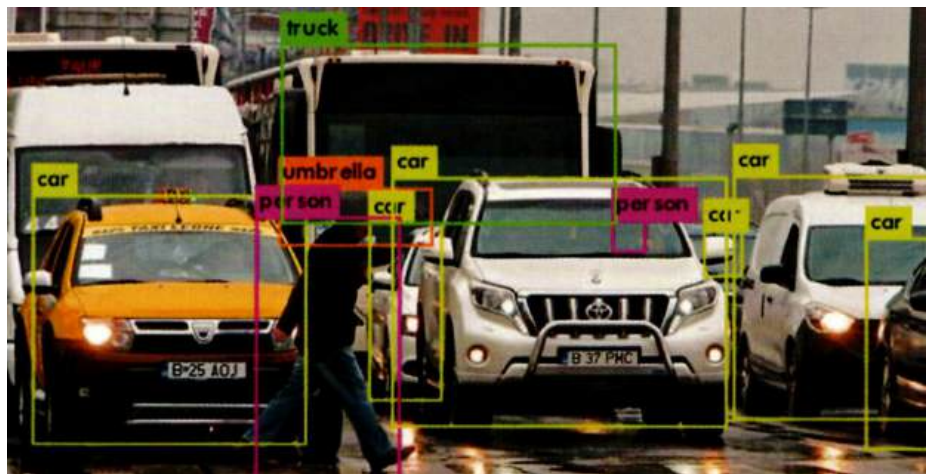


1.3.2 딥러닝 학습 알고리즘

- 활용 분야에 따라 분류되는 것들
 - 지도 학습
 - 비지도 학습
 - 전이 학습 (Transfer Learning)

지도 학습

- 이미지 분류 → 컴퓨터 비전 기술
 - 많이 사용 하는 것 :
 - 합성곱 신경망 (Convolutional Neural Network, CNN)
 - 목적에 따라 이미지 분류, 이미지 인식, 이미지 분할로 분류할 수 있음
 - 이미지 분류 : 이미지 데이터를 유사한 것 끼리 분류할 때 사용
 - 이미지 인식 : 사진 안에 있는 사물의 종류 인식 → 의료 이미지에서 질병 식별



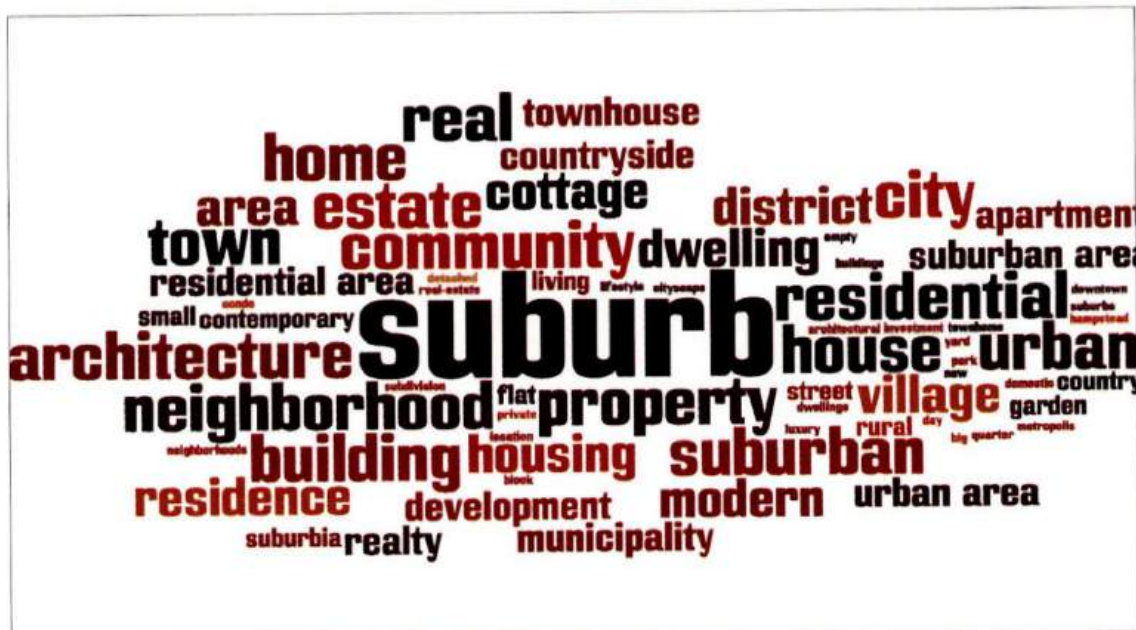
- 이미지 분할 : 영상에서 사물 or 배경 등 객체간 영역을 픽셀 단위로 구분하는 기술
 - X-ray, CT, MRI 등 의료 영상들에서 분할된 이미지 정보로 질병 진단
- 순환 신경망 (Recurrent Neural Network, RNN)
 - 시계열 데이터 분류 (예: 주식 데이터)

- 단점 : 역전파 과정에서 기울기 소멸 문제 → gate 3 개 추가함 → LSTM(Long Short-Term Memory)
 - 망각 게이트(과거 정보 잊기 위한)
 - 입력 게이트(현재 정보 기억하기 위한)
 - 출력 게이트(최종 결과를 위한 게이트)

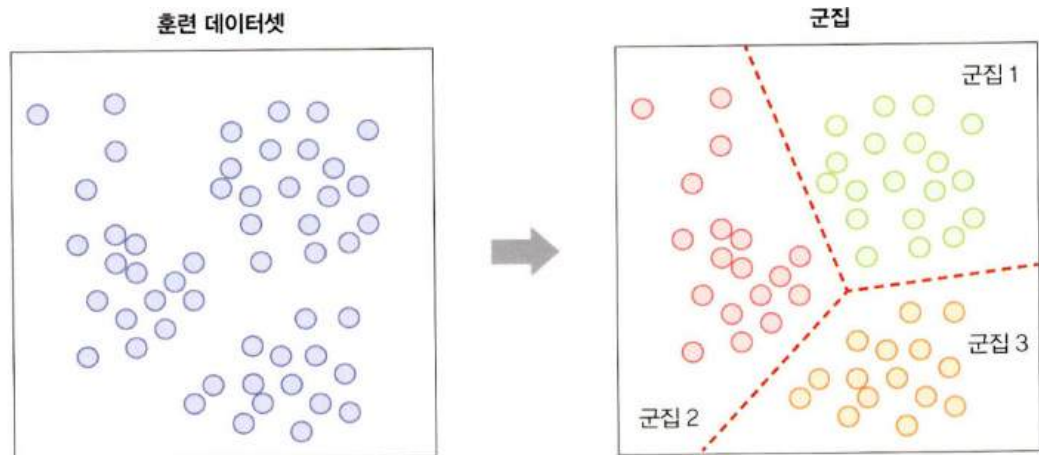
비지도 학습

- 종료
 - 워드 임베딩
 - 군집
- 워드 임베딩 : 컴퓨터가 이해 가능한 자연어로 변환
 - 단어를 벡터로 표현 (Word2Vec, GloVe)
 - 예 : 번역

♥ 그림 1-17 워드 임베딩을 이용한 워드 클라우드



- 군집 : 아무런 정보 없는 상태에서 데이터를 분류하는 방법
 - 한 cluster 안의 데이터는 비슷하게 구성 (다른 cluster의 데이터와 구분 되도록)



전이 학습(Transfer Learning)

- 사전에 학습 완료된 모델(pre-trained model)을 가지고 원하는 학습에 미세 조정 기법이 이용해서 학습시키는 방법
 - 사전 학습 모델
 - 풀고자 하는 문제와 비슷함
 - 많은 데이터로 학습 되어 있는 모델
 - VGG, Inception, MobileNet
 - 사전 학습 모델 활용법:
 - 특성 추출
 - 미세 조정

강화 학습

- 머신 러닝과 동일

지도 학습, 비지도 학습, 전이 학습, 강화 학습 표

구분	유형	알고리즘
지도 학습(supervised learning)	이미지 분류	• CNN • AlexNet • ResNet
	시계열 데이터 분석	• RNN • LSTM
비지도 학습 (unsupervised learning)	군집 (clustering)	• 가우시안 혼합 모델(Gaussian Mixture Model, GMM) • 자기 조직화 지도(Self-Organizing Map, SOM)
	차원 축소	• 오토인코더(AutoEncoder) • 주성분 분석(PCA)
전이 학습(transfer learning)	전이 학습	• 버트(BERT) • MobileNetV2
강화 학습(reinforcement learning)	—	마르코프 결정 과정(MDP)



2장 : 실습 환경 설정과 파이토치 기초

2.1 파이토치 개요

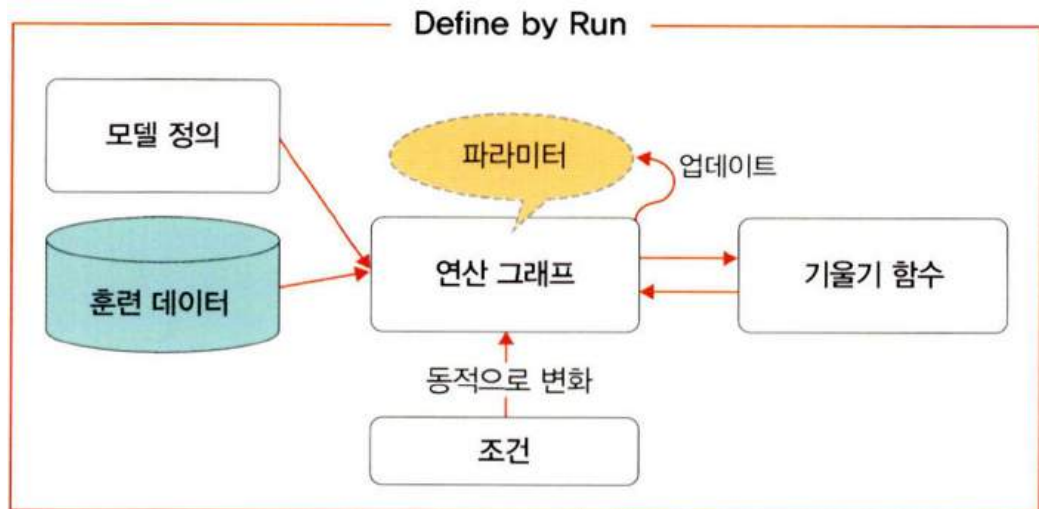
파이토치(PyTorch)

- 딥러닝 프레임워크로 루아 언어로 개발된 Torch의 파이썬 버전
- 파이썬 기반의 과학 연산 패키지
 - 넘파이 대체 + GPU 이용한 연산이 필요한 경우
 - 유연성 + 속도를 제공한 딥러닝 연구 플랫폼이 필요한 경우
- 간결함 + 빠른 구현성

2.1.1 파이토치 특징 및 장점

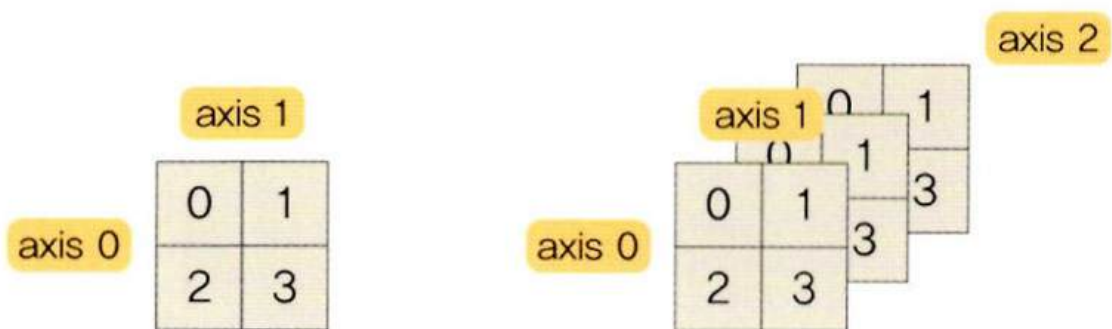
- GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 프레임워크
 - GPU(Graphic Processing Unit)
 - 연산 속도 빠르게 하는 역할
 - 기울기 계산할 때의 미분 사용 할때 빠름
 - CUDA, cuDNN이라는 API 통해 GPU 연산에 사용
 - 병렬 연산
 - CPU보다 훨씬 빠름
 - 텐서(Tensor)
 - 파이토치의 데이터 형태
 - 단일 데이터 형식으로된 자료들의 다차원 행렬
 - 간단한 명령어 (변수 뒤에 .cuda() 추가)하면 GPU로 연산 가능
 - 동적 신경망
 - 훈련 반복할 때마다 네트워크 변경이 가능한 신경망

- 은닉층 추가 or 제거
- 연산 그래프 정의 하면 값 초기화되는 "Define by Run" 사용



벡터, 행렬, 텐서

- 인공지능에서 데이터는 벡터로 표현됨 → [1.0, 1.1, 1.2] (1차원 배열)
- 행렬 : 행과 열 (2차원 배열)
- 텐서 : 3차원 이상의 배열
- 차이



- 1차원 축(행) = axis 0 = 벡터
- 2차원 축(열) = axis 1 = 행렬
- 2차원 축(채널) = axis 2 = 텐서
- 행렬(Matrix)은 데이터 레코드들의 집합
 - 하나의 데이터 레코드를 벡터로 나타내면, 보통 열벡터 형태로 표현

$$x_1 = \begin{bmatrix} 1.1 \\ 2.7 \\ 3.3 \\ 0.2 \end{bmatrix} \quad x_2 = \begin{bmatrix} 4.5 \\ 1.2 \\ 0.7 \\ 3.5 \end{bmatrix}$$

- $x_1, x_2 \rightarrow$ 각각 하나의 데이터 레코드
- 여러 레코드를 행렬로 표현
 - 각 레코드가 행(row)에 위치

$$X = \begin{bmatrix} 1.1 & 2.7 & 3.3 & 0.2 \\ 4.5 & 1.2 & 0.7 & 3.5 \end{bmatrix}$$

- 첫 번째 행 $\rightarrow x_1$, 두 번째 행 $\rightarrow x_2$
- 행렬의 일반적인 형태
 - $x_{ij} \rightarrow i$ 행, j 열의 원소

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}$$

- 텐서(Tensor)는 행렬의 일반화
 - 텐서는 행렬을 더 일반화한 다차원 배열
 - 예:
 - 스칼라(Scalar) = 0차원 텐서 (숫자 하나)
 - 벡터(Vector) = 1차원 텐서
 - 행렬(Matrix) = 2차원 텐서
 - 그 이상의 차원(3D, 4D ...) = 고차원 텐서
- 파이토치에서 텐서를 만드는 방법:

```
import torch
torch.tensor([[1., -1.], [1., -1.]])
```

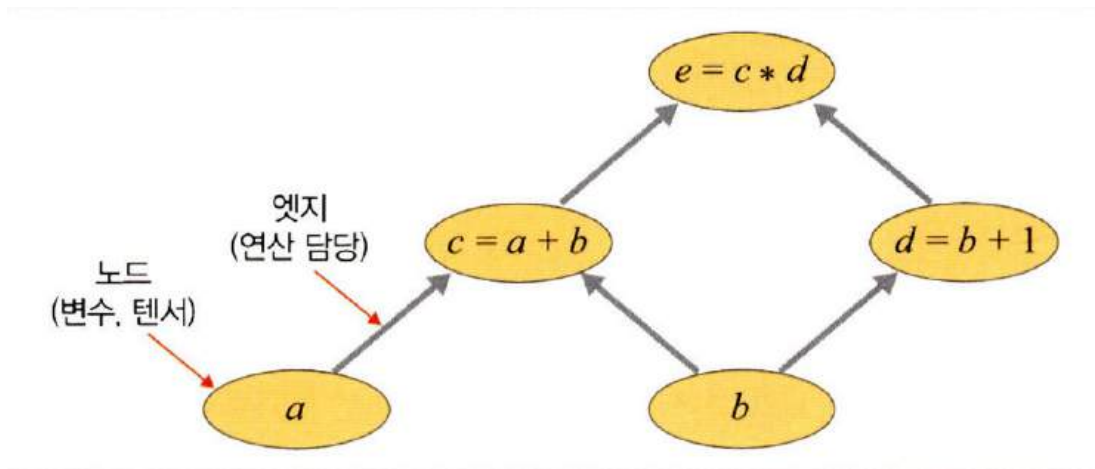
- 생성된 텐서는 2×2 행렬(텐서)로 표현

```
tensor([[ 1., -1.],
        [ 1., -1.]])
```

연산 그래프

- 연산 그래프는 방향성이 있는 그래프로,

- 노드(node) → 변수(예: 텐서)
- 엣지(edge) → 연산(예: 더하기, 곱하기)



- 입력: a, b
- 중간 계산: c, d
- 최종 출력: e
- 신경망에서 연산 그래프를 쓰는 이유
 - 신경망 학습에서 손실 함수의 기울기(gradient)를 계산해 가중치와 바이어스를 업데이트해야 함
 - 연산 그래프는 출력이 입력과 파라미터에 어떻게 의존하는지를 기록
 - 이를 통해 역전파(Backpropagation , 경사 하강법)가 가능해짐

파이토치의 장점

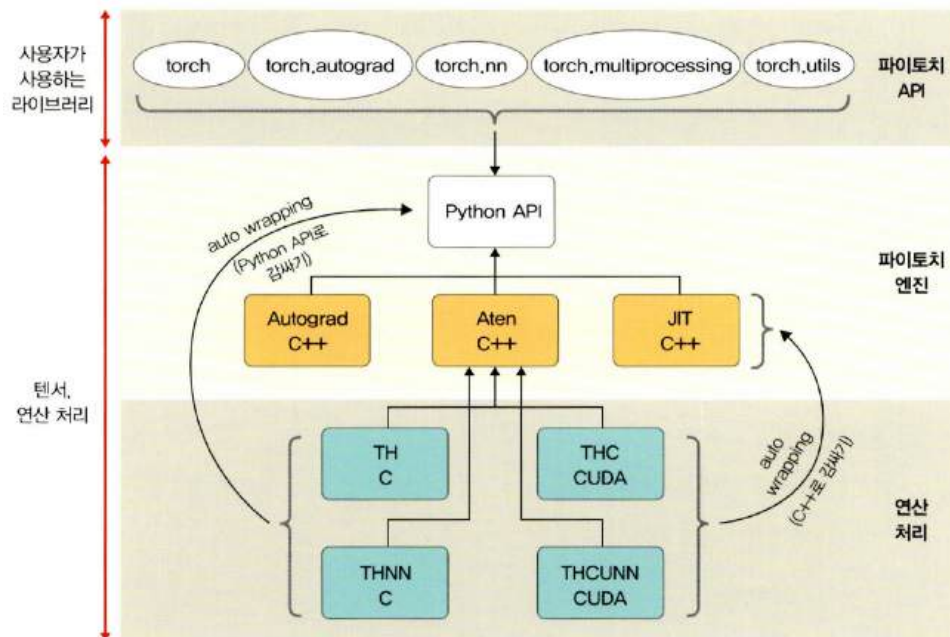
- 단순함 : 파이선 환경과 쉽게 통합, 디버깅 간결
- 성능 : 낮은 CPU 활용
- 직관적인 인터페이스

2.1.2 파이토치의 아키텍처

- 3개의 계층
 1. 파이토치 API
 2. 파이토치 엔진
 - 다차원 텐서 및 자동 미분 처리

3. 텐서 연산

- Architecture



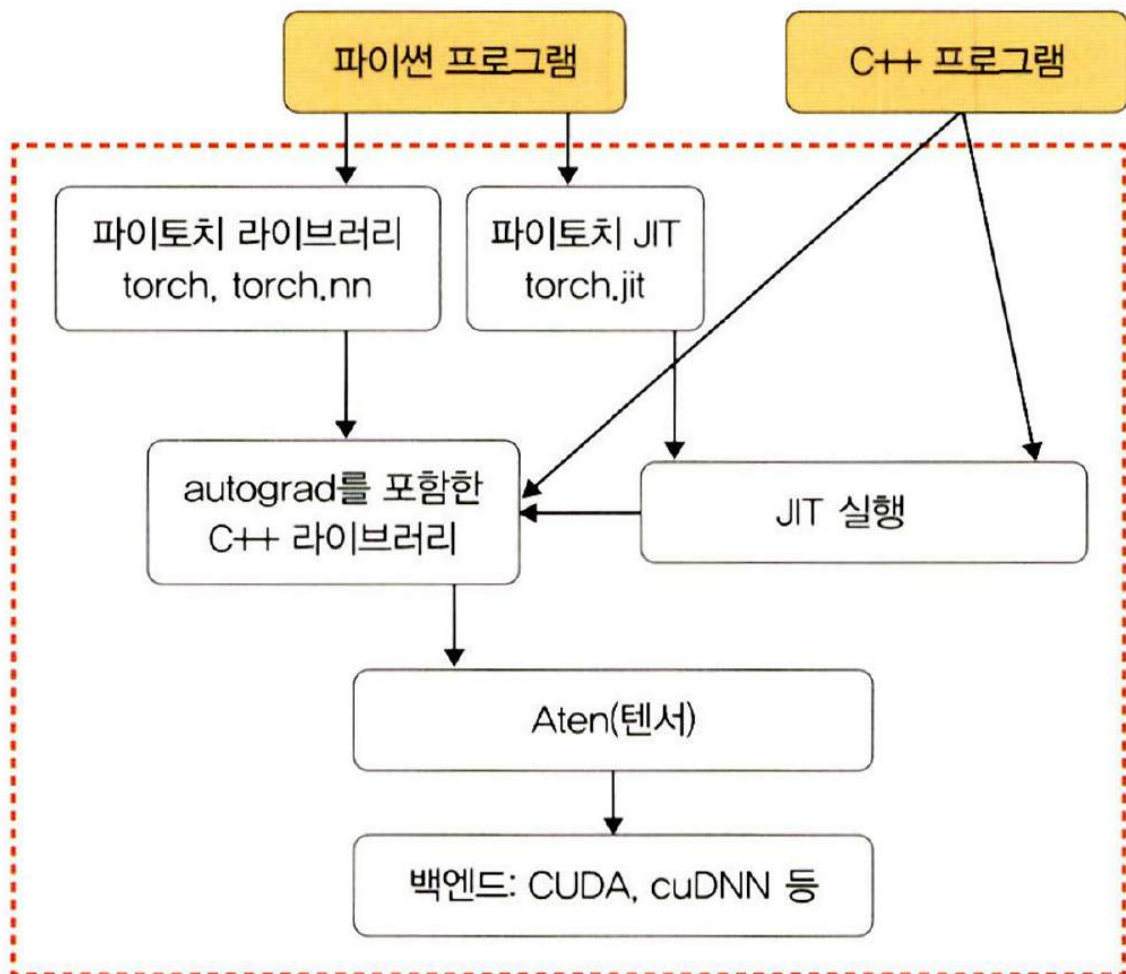
파이토치 API

- 최상위 계층: (사용자가 직접 사용하는 라이브러리)
- Python 코드에서 직접 import 해서 쓰는 부분
- 역할:
 - 사용자가 쉽게 이해하고 쓸 수 있는 API 제공
 - 텐서를 이용한 연산 및 신경망 구축/훈련을 할 수 있도록 도와줌
 - 실제 연산은 수행하지 않고, C++로 작성된 파이토치 엔진에 작업을 전달하는 역할
- 주요 파이토치 API 패키지
 - `torch` → GPU 지원 텐서 패키지
 - 다차원 텐서 연산 제공
 - CPU뿐 아니라 GPU에서도 연산 가능
 - 빠르고 대규모 데이터 계산을 지원
 - `torch.autograd` → 자동 미분 패키지 (역전파)
 - 파이토치의 자동 미분 엔진(`autograd`)을 제공

- 다른 프레임워크와 달리, 파이토치는 동적 그래프(Define-by-Run) 방식을 사용
 - 실행 시점에 연산 그래프가 즉시 생성됨
 - 역전파 시 효율적으로 기울기(gradient) 계산 가능
 - 모델 구조를 유연하게 변경해도 재구축 필요 없이 즉시 적용 가능
- torch.nn → 신경망 구축 및 훈련 패키지
 - 신경망을 쉽게 만들고 학습할 수 있는 모듈 제공
 - 합성곱 신경망(CNN), 순환 신경망(RNN), 정규화, 활성화 함수 등 기본 블록 포함
 - 복잡한 모델도 모듈을 조합하여 간단히 구축 가능
- torch.multiprocessing → 파이썬 멀티프로세싱 패키지
 - 여러 프로세스 간에 텐서를 공유할 수 있도록 지원
 - 서로 다른 프로세스에서 같은 텐서 데이터를 접근/사용 가능
 - 병렬 학습이나 대규모 분산 처리에서 유용
- torch.utils → DataLoader 및 유틸리티 패키지
 - 데이터 처리와 기능 제공
 - 주요 모듈:
 - torch.utils.data.DataLoader → 데이터를 효율적으로 불러오고 배치 처리
 - torch.utils.bottleneck → 성능 병목 현상 점검
 - torch.utils.checkpoint → 메모리 절약을 위해 중간 계산 결과 저장/재사용

파이토치 엔진

- 파이토치 엔진 라이브러리는 C++로 작성되어 있지만, Python API로 래핑(wrapping)되어 제공됨
- 사용자는 Python에서 간단하게 모델을 작성하고 텐서를 다루지만, 실제 계산은 C++/CUDA가 고속으로 처리함
- 파이토치 엔진



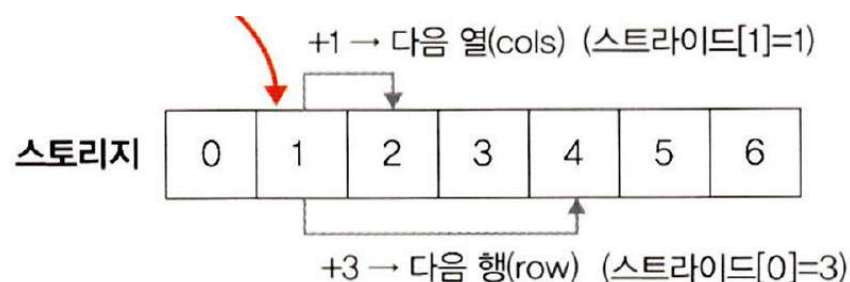
- 파이썬 프로그램
 - torch, torch.nn, torch.jit 같은 라이브러리를 사용해 모델 작성
- C++ 프로그램
 - 필요하다면 직접 C++ 백엔드 호출도 가능
- C++ 라이브러리 (autograd 포함)
 - 텐서 연산과 자동 미분 기능 구현
- JIT 실행
 - TorchScript 코드 실행을 통해 성능 최적화
- Aten (텐서 라이브러리)
 - 텐서 생성 및 연산을 처리하는 핵심 라이브러리
- 백엔드 (CUDA, cuDNN 등)
 - 실제 CPU/GPU에서 수치 계산 수행

연산 처리

- 연산 처리 계층
 - 파이토치의 가장 아래 계층은 C 또는 CUDA 로 구현
 - TH (Torch CPU용), THC (Torch CUDA/GPU용) 같은 패키지가 실제 계산을 담당
 - 다차원 텐서 연산과 효율적인 데이터 구조를 제공
 - 사용자가 torch, torch.nn 등에서 호출하는 연산은 최종적으로 이 계층에서 처리
- 텐서를 메모리에 저장하기
 - 텐서는 메모리에 저장될 때 실제로는 1차원 배열(Flat memory) 로 저장됨
 - 이 1차원 배열을 스토리지(storage) 라고 부름
 - 다차원 텐서를 스토리지와 연결하기 위해 두 가지 개념:
 - 오프셋(offset) → 스토리지에서 텐서의 첫 번째 요소가 위치한 인덱스 (index in storage where first element is located)
 - 스트라이드(stride) → 각 차원에서 다음 요소로 이동하기 위해 건너뛰어야 하는 스토리지의 칸 수(how many storage elements to skip to move to the next element along each dimension)
 - 예: 행렬과 전치 행렬

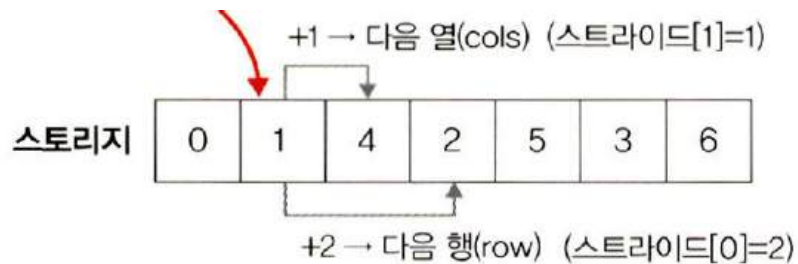


- A



- 1차원 storage : [0,1,2,3,4,5,6]
- shape : (2,3)
- 오프셋(offset) = 1 (1번 인덱스에서 시작)
- 스트라이드(stride) = (3, 1)
 - 열 방향으로 이동 → 1칸 건너뛰
 - 행 방향으로 이동 → 3칸 건너뛰

■ A^T



- 1차원 storage : [0,1,4,2,5,3,6]
- shape : (3,2)
- 오프셋(offset) = 1
- 스트라이드(stride) = (2, 1)
 - 열 방향으로 이동 → 1칸 건너뛰
 - 행 방향으로 이동 → 2칸 건너뛰

2.2 파이토치 기본 문법

2.2.1 텐서 다루기

- 파이토치에서 텐서는 가장 기본이 되는 데이터 구조
- NumPy의 ndarray와 비슷함
 - GPU 연산까지 지원한다는게 차이

텐서 생성 및 변환

- 텐서 생성

```
import torch
```

```
print(torch.tensor([[1, 2], [3, 4]]))          # 2차원 텐서 생성 (CPU)
print(torch.tensor([[1, 2], [3, 4]], device="cuda:0")) # 텐서 생성 (GPU)
print(torch.tensor([[1, 2], [3, 4]], dtype=torch.float64)) # dtype 지정하여 생성
```

- 결과

```
tensor([[1, 2],
        [3, 4]]) #int

tensor([[1., 2.],
        [3., 4.]], dtype=torch.float64) #float
```

- Numpy 배열로 변환

```
temp = torch.tensor([[1, 2], [3, 4]])
print(temp.numpy()) # 텐서를 ndarray로 변환

temp = torch.tensor([[1, 2], [3, 4]], device="cuda:0")
print(temp.to("cpu").numpy()) # GPU 텐서를 CPU로 옮긴 뒤 ndarray로 변환
```

- 결과

```
[[1 2]
 [3 4]]

[[1 2]
 [3 4]]
```

텐서의 인덱스 조작

- 텐서의 자료형 (Data Types)

파이토치에서 자주 쓰이는 텐서 타입:

- torch.FloatTensor → 32비트 부동 소수점 (float32)

- torch.DoubleTensor → 64비트 부동 소수점 (float64)
- torch.LongTensor → 64비트 정수 (int64)
- 텐서 인덱싱

```
import torch

temp = torch.FloatTensor([1, 2, 3, 4, 5, 6, 7]) # 1차원 텐서 생성
print(temp[0], temp[1], temp[-1]) # 인덱스로 접근
print('-----')
print(temp[2:5], temp[4:-1]) # 슬라이싱으로 접근
```

- 결과

```
tensor(1.) tensor(2.) tensor(7.)
-----
tensor([3., 4., 5.]) tensor([5., 6.])
```

텐서 연산 및 차원 조작

- 사칙연산
 - 넘파이 배열처럼 다양한 수학 연산을 지원
 - 연산은 CPU와 GPU 모두에서 수행할 수 있음
 - 중요: 두 텐서의 자료형(dtype) 이 같아야 연산이 가능합니다.
 - 예: FloatTensor와 DoubleTensor는 직접 연산 불가

```
v = torch.tensor([1, 2, 3]) # 길이 3인 벡터
w = torch.tensor([3, 4, 6]) # 길이 3인 벡터
print(w - v)
```

- 결과

```
tensor([2, 2, 3])
```

- 차원 조작
 - 딥러닝에서 텐서의 형태(shape)를 바꾸는 일이 매우 자주 발생

- 예: 이미지 데이터를 평탄화(flatten) 하거나, 배치(batch) 단위로 변환할 때
- 주요 메서드
 - `.view()` → 모양 변경 (넘파이 reshape과 유사)
 - `.t()` → 전치(Transpose), 행렬의 행과 열 교환
 - `.transpose()` → 여러 차원의 순서 변경
 - `.stack()`, `.cat()` → 텐서 결합
- 예: `.view()`로 형태 변경

```
temp = torch.tensor([
    [1, 2],
    [3, 4]
]) # shape = (2, 2)

print(temp.shape)      # torch.Size([2, 2])
print(temp.view(4, 1)) # 2×2 → 4×1
print(temp.view(-1))   # 2×2 → (4,) 1차원 벡터로 평탄화
print(temp.view(1, -1)) # 2×2 → (1, 4)
print(temp.view(-1, 1)) # 2×2 → (4, 1)
```

- 1은 해당 차원을 자동으로 추론하라는 의미.
- `temp`의 원소 개수는 4개이므로:
 - `view(-1)` → (4,)로 평탄화
 - `view(1, -1)` → (1, 4)로 변환
 - `view(-1, 1)` → (4, 1)로 변환
- 결과

```
torch.Size([2, 2])

tensor([[1],
        [2],
        [3],
        [4]])

tensor([1, 2, 3, 4])
```

```
tensor([[1, 2, 3, 4]])
```

```
tensor([[1],  
        [2],  
        [3],  
        [4]])
```

2.2.2 데이터 준비

단순하게 파일을 불러와서 사용

- 데이터가 CSV, JSON, PDF 형태일 경우 → pandas 라이브러리로 불러올 수 있습니다.
- 이후 데이터를 PyTorch Tensor로 변환해서 사용합니다.
- 예 :

```
import pandas as pd  
import torch  
  
data = pd.read_csv('./class2.csv') # CSV 불러오기  
  
x = torch.from_numpy(data['x'].values).unsqueeze(dim=1).float()  
y = torch.from_numpy(data['y'].values).unsqueeze(dim=1).float()
```

- pandas로 CSV 읽기
- .values → 넘파이 배열 추출
- torch.from_numpy() → 텐서로 변환
- .unsqueeze(dim=1) → 차원 추가 (예: (n,) → (n,1))
- .float() → 자료형을 float으로 변환

커스텀 데이터셋을 만들어서 사용

- torch.utils.data.Dataset
 - 데이터가 너무 크면 한 번에 메모리에 올리기 비효율적 → Dataset 클래스를 직접 정의해서 샘플 단위로 불러옴

- 예:

```
from torch.utils.data import Dataset

class CustomDataset(Dataset):
    def __init__(self, csv_file):
        self.label = pd.read_csv(csv_file) # CSV 파일 읽기

    def __len__(self):
        return len(self.label) # 데이터 전체 크기 반환

    def __getitem__(self, idx):
        sample = torch.tensor(self.label.iloc[idx, 0:3]).int()
        label = torch.tensor(self.label.iloc[idx, 3]).int()
        return sample, label
```

- `__init__` → 데이터셋 초기화
- `__len__` → 전체 데이터 크기 반환
- `__getitem__` → 인덱스 기반으로 하나의 샘플(feature + label) 반환
- `torch.utils.data.DataLoader`
 - Dataset을 감싸서 배치 처리, 셔플, 반복(iteration) 기능 제공

```
from torch.utils.data import DataLoader

tensor_dataset = CustomDataset('./covtype.csv')
dataset = DataLoader(tensor_dataset, batch_size=4, shuffle=True)
```

- 데이터셋을 `batch_size=4`로 나눠 가져옴
- `shuffle=True` → 매 epoch마다 데이터 순서 랜덤하게 섞음
- `DataLoader` 동작 과정 (도식 참고)
 1. 전체 데이터셋 (A, B, C)
 2. 무작위로 섞기 (예: B, A, C)
 3. 큐(queue)에 적재
 4. 배치 단위로 나눔 → 배치1, 배치2, ...

- 예:

```
for i, data in enumerate(dataset, 0):  
    batch = data[0]  
    print(i, batch.size())
```

- 결과

```
0 torch.Size([4, 3])  
1 torch.Size([4, 3])  
2 torch.Size([4, 3])
```

- 각 배치는 4개의 샘플, 각 샘플은 feature 3개

파이토치에서 제공하는 데이터셋 사용

- PyTorch는 torchvision.datasets 안에 유명한 데이터셋 제공 (MNIST 등)
- 예:

```
import torchvision.transforms as transforms  
from torchvision.datasets import MNIST  
  
# 전처리 -> 텐서 변환 + 정규화  
mnist_transform = transforms.Compose([  
    transforms.ToTensor(),  
    transforms.Normalize((0.5,), (1.0,))  
])  
  
download_root = './chap02/data/MNIST_DATASET'  
  
train_dataset = MNIST(download_root, transform=mnist_transform, train=True, download=True)  
valid_dataset = MNIST(download_root, transform=mnist_transform, train=False, download=True)  
test_dataset = MNIST(download_root, transform=mnist_transform, train=False, download=True)
```

- ToTensor() → 이미지를 Tensor로 변환

- `Normalize((0.5,), (1.0,))` → 평균 0.5, 표준편차 1.0으로 정규화
- `train=True/False` → 학습/검증/테스트 데이터셋 구분

2.2.3 모델 정의

계층, 모듈, 모델의 개념

- 계층(Layer): 신경망의 한 층 (예: `nn.Linear`, `nn.Conv2d`)
- 모듈(Module): 하나 이상의 계층을 포함할 수 있는 단위
- 모델(Model): 최종적으로 학습시키려는 전체 네트워크. 하나의 모듈이 모델이 될 수도 있고, 여러 모듈이 모여 모델이 되기도 함

모델 정의 방법

- 단순 모델 정의

```
import torch.nn as nn
model = nn.Linear(in_features=1, out_features=1, bias=True)
```

- 입력 1개 → 출력 1개인 단순 선형 모델

- `nn.Module`을 상속받아 정의

```
class MLP(nn.Module):
    def __init__(self, inputs):
        super(MLP, self).__init__()
        self.layer = nn.Linear(inputs, 1)    # 계층 정의
        self.activation = nn.Sigmoid()       # 활성화 함수 정의

    def forward(self, X):
        X = self.layer(X)                    # 선형 변환
        X = self.activation(X)                # 활성화 적용
        return X
```

- `__init__`에서 계층 정의, `forward()`에서 연산 순서를 지정
- `nn.Sequential`로 정의

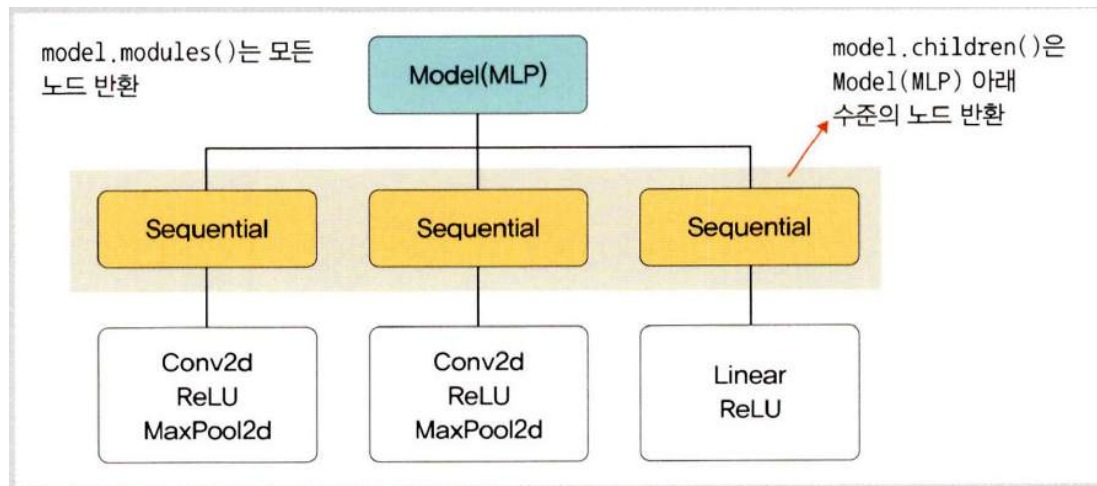
```

class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=5),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2)
        )
        self.layer2 = nn.Sequential(
            nn.Conv2d(64, 30, kernel_size=5),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2)
        )
        self.layer3 = nn.Sequential(
            nn.Linear(30*5*5, 10),
            nn.ReLU(inplace=True)
        )

    def forward(self, x):
        x = self.layer1(x)
        x = self.layer2(x)
        x = x.view(x.shape[0], -1) # Flatten
        x = self.layer3(x)
        return x

```

- nn.Sequential은 여러 계층을 순서대로 묶어 간단히 정의할 수 있음
- model.children() vs model.modules()
 - model.children() → 같은 수준(Level) 의 하위 모듈만 반환
 - model.modules() → 모델에 포함된 모든 모듈(중첩 포함) 반환



- children()은 Sequential 블록만 반환
- modules()는 그 안의 Conv, ReLU, Pooling까지 전부 반환
- 함수형으로 정의

```
def MLP(in_features=1, hidden_features=20, out_features=1):
    hidden = nn.Linear(in_features, hidden_features)
    activation = nn.ReLU()
    output = nn.Linear(hidden_features, out_features)
    net = nn.Sequential(hidden, activation, output)
    return net
```

2.2.4 모델의 파라미터 정의

손실 함수 (Loss Function)

- 모델의 출력값과 실제 값(정답) 사이의 오차를 측정하는 함수.
- 오차가 클수록 손실 함수 값이 커지고, 이를 줄이는 방향으로 모델이 학습됨.
- 자주 쓰이는 손실 함수:
 - BCELoss: 이진 분류 (Spam/Not Spam 같은 경우)
 - CrossEntropyLoss: 다중 클래스 분류 (예: 숫자 0~9 분류)
 - MSELoss: 회귀 문제 (예: 집값 예측)

옵티마이저 (Optimizer)

- 손실 함수를 최소화하도록 모델의 파라미터(가중치, bias)를 업데이트하는 역할.

- 주요 기능:
 - `optimizer.step()` : 파라미터 업데이트 수행
 - `optimizer.zero_grad()` : 기울기(`gradient`) 초기화
 - 파라미터별로 다른 학습률 지정 가능
- 대표적인 옵티마이저:
 - SGD (확률적 경사 하강법)
 - Adam, RMSProp, Adagrad, Adadelta 등

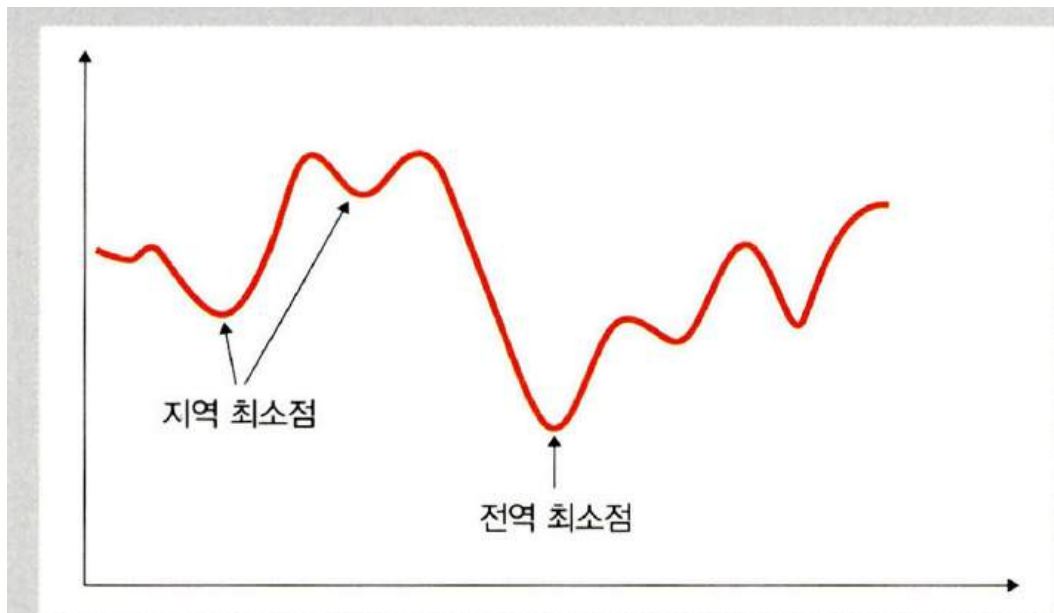
학습률 스케줄러 (Learning Rate Scheduler)

- 학습률을 고정하지 않고, 에포크 진행에 따라 점점 줄이거나 변화시키는 방법.
- 이유: 학습 초반에는 빠르게 학습 → 후반에는 세밀하게 조정하기 위해.
- 주요 스케줄러:
 - LambdaLR: 사용자 정의 함수(`lambda`)에 따라 학습률 조정
 - StepLR: 일정 step마다 학습률 감소
 - MultiStepLR: 여러 단계마다 학습률 감소
 - ExponentialLR: 지수적으로 감소
 - CosineAnnealingLR: 코사인 곡선 형태로 변화
 - ReduceLROnPlateau: 학습이 잘 안될 때 학습률을 줄임

지표 (Metrics)

- 모델 성능을 평가하기 위한 척도.
- 예: 정확도(Accuracy), F1 Score, RMSE 등.

전역 최소점 vs 지역 최소점



- 전역 최소점(Global Minimum): 손실 함수의 가장 작은 값 → 최적의 해
- 지역 최소점(Local Minimum): 전역 최소점은 아니지만, 주변보다 작은 값
- 신경망 학습은 지역 최소점에 빠지지 않고 전역 최소점에 가깝게 가는 것이 목표

예시 코드

```
from torch.optim import SGD, lr_scheduler
import torch.nn as nn

# 손실 함수 정의
criterion = nn.MSELoss()

# 옵티마이저 정의 (SGD, 학습률 0.01, 모멘텀 0.9)
optimizer = SGD(model.parameters(), lr=0.01, momentum=0.9)

# 학습률 스케줄러 정의 (매 epoch마다 0.95배 감소)
scheduler = lr_scheduler.LambdaLR(optimizer, lr_lambda=lambda epoch: 0.95 ** epoch)

# 학습 루프
for epoch in range(1, 101): # 100 에포크 학습
    for x, y in dataloader: # 배치 단위 학습
        optimizer.zero_grad() # 기울기 초기화
        loss = criterion(model(x), y) # 손실 계산
```

<code>loss.backward()</code>	# 역전파
<code>optimizer.step()</code>	# 파라미터 업데이트
<code>scheduler.step()</code>	# 학습률 업데이트

2.2.5 모델 훈련

- 모델을 학습시킨다는 것은 함수 $y = wx + b$ 에서 w 와 b 의 최적값을 찾는 과정입니다.
- 처음에는 임의의 값으로 시작 → 오차를 줄여가며 전역 최소점(global minimum) 에 이를 때까지 파라미터(w, b)를 계속 수정합니다.

훈련 절차

PyTorch에서 모델을 훈련할 때 핵심 단계는 다음과 같습니다:

1. `optimizer.zero_grad()`
 - 기울기(`gradient`)를 0으로 초기화.
 - 이유: PyTorch는 기본적으로 기울기를 누적(`accumulate`) 하기 때문에, 매번 새로운 학습 단계 전에 초기화해야 함.
2. 순전파(Forward pass)
 - 입력 데이터를 모델에 통과시켜 출력 계산
 - `output = model(input)`
3. 손실(Loss) 계산
 - 예측값(`output`)과 실제값(`target`) 차이를 계산
 - `loss = loss_fn(output, target)`
4. 역전파(Backward pass)
 - `loss.backward()` 호출 → 기울기 자동 계산
5. 파라미터 업데이트
 - `optimizer.step()` → 계산된 기울기를 이용해 가중치(w, b)를 업데이트

예시 코드

```
for epoch in range(100):          # 총 100번 학습 반복
    yhat = model(x_train)         # 순전파: 예측값 계산
    loss = criterion(yhat, y_train) # 손실 함수 계산
```

```
optimizer.zero_grad()      # 기울기 초기화 (중첩 방지)
loss.backward()            # 역전파로 기울기 계산
optimizer.step()           # 파라미터 업데이트
```

2.2.6 모델 평가

모델을 훈련한 후, 주어진 테스트 데이터셋을 사용하여 성능을 평가함

평가는 크게 두 가지 방식:

1. 함수(functional) 를 이용하는 방법
2. 모듈(class) 을 이용하는 방법

함수로 모델 평가하기

- torchmetrics 라이브러리를 설치

```
pip install torchmetrics
```

```
import torch
import torchmetrics

preds = torch.randn(10, 5).softmax(dim=-1) # 10개 샘플, 5개 클래스에 대
한 예측 확률
target = torch.randint(5, (10,))           # 실제 정답(라벨)

# accuracy 계산
acc = torchmetrics.functional.accuracy(preds, target)
```

- functional.accuracy 를 이용해서 정확도를 계산

모듈로 모델 평가하기

모듈을 초기화해서 여러 배치(batch)에 대해 평가한 뒤, 전체 평균 정확도를 낼 수도 있음

```
import torch
import torchmetrics
```

```
metric = torchmetrics.Accuracy() # 모델 평가용 정확도 metric 초기화

n_batches = 10
for i in range(n_batches):
    preds = torch.randn(10, 5).softmax(dim=-1) # 예측값
    target = torch.randint(5, (10,)) # 실제값
    acc = metric(preds, target) # 배치별 정확도
    print(f"Accuracy on batch {i}: {acc}")

# 전체 배치에 대한 최종 정확도
acc = metric.compute()
print(f"Accuracy on all data: {acc}")
```

- `metric(preds, target)` → 배치 단위 평가
- `metric.compute()` → 전체 데이터에 대해 평가 (누적된 결과 기반)

`torchmetrics` 말고 사이킷런(`sklearn`) 에서 제공하는

- `confusion_matrix` (혼동 행렬)
- `accuracy_score` (정확도)
- `classification_report` (정밀도, 재현율, F1-score)

등을 활용할 수도 있음

2.2.7 훈련 과정 모니터링

- 모델을 학습하다 보면 각 파라미터 값이 어떻게 변하는지 확인하기 어려움 → 이때 TensorBoard를 사용하면 손쉽게 시각화하여 확인할 수 있음

텐서보드 사용 과정

- 텐서보드 설치

```
pip install tensorboard
```

- `SummaryWriter` 설정

```
from torch.utils.tensorboard import SummaryWriter
```

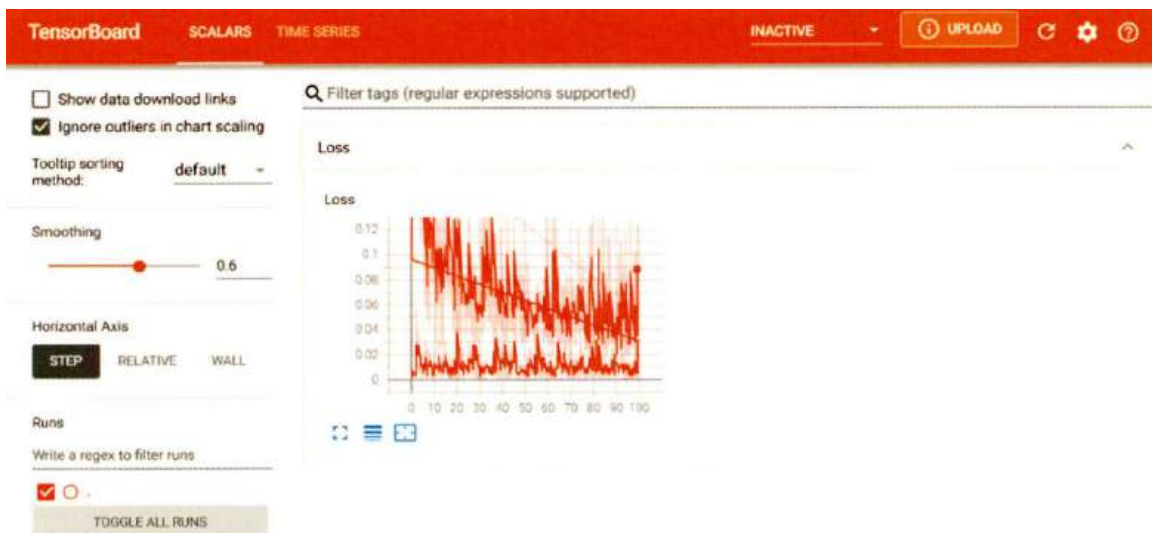
```
writer = SummaryWriter("../chap02/tensorboard")
```

→ 훈련 과정에서 기록할 값들이 저장될 위치 지정.

- 훈련 과정에서 값 기록
 - `writer.add_scalar("Loss", loss, epoch)` : 각 epoch마다 손실(loss) 값 기록
 - 학습이 끝나면 `writer.close()` 호출하여 종료.
- 텐서보드 실행

```
tensorboard --logdir=../chap02/tensorboard --port=6006
```

→ 브라우저에서 <http://localhost:6006> 접속하여 시각화 확인



예제 코드 (훈련 기록)

```
for epoch in range(num_epochs):
    model.train() # 학습 모드
    for i, (x, y) in enumerate(dataloader):
        outputs = model(x)
        loss = criterion(outputs, y)

        writer.add_scalar("Loss", loss, epoch) # 손실 기록

    optimizer.zero_grad()
```

```
loss.backward()
optimizer.step()
```

model.train() vs model.eval()

- model.train() : 학습 모드. Dropout, BatchNorm 등이 활성화됨.
- model.eval() : 평가 모드. Dropout 비활성화, 전체 노드 사용 → 검증/테스트 시 사용.

특히 model.eval()에서는 with torch.no_grad():를 같이 사용해야 함.

→ 이유: 평가 시에는 기울기(gradient) 계산이 필요 없으므로 메모리와 연산 시간을 절약

2.3 실습 환경 설정

2.3.1 아나콘다(Anaconda) 설치

- 다운로드 → 공식 사이트에서 OS 맞는 버전 받기.
- 설치 시작 → Next → I Agree → Just Me 선택.
- 경로 설정 → 기본값 두고 Next.
- 옵션 선택 → PATH 등록 + 기본 Python 설정 체크 → Install.
- 설치 완료 → Finish.
- 환경 변수 확인 → 자동 등록 되었는지 확인.

2.3.2 가상 환경 생성 및 파이토치 설치

가상 환경 생성하기

1. Anaconda Prompt 실행

윈도우 시작 메뉴에서 Anaconda Prompt를 실행.

2. 가상 환경 만들기

```
conda create -n torch_book python=3.9.0
```

→ torch_book이라는 이름의 가상 환경 생성. (파이썬 3.9 권장)

3. 가상 환경 목록 확인

```
conda env list
```

→ 생성된 가상 환경 확인 가능.

4. 가상 환경 활성화

```
activate torch_book
```

5. 커널 설치 및 연결

```
conda install ipykernel  
ipython kernel install --name tf2_book --user  
jupyter notebook
```

→ 주피터 노트북에서 해당 가상환경 사용 가능.

파이토치 설치하기

- Anaconda Prompt에서 다음 명령어 실행 (CPU 기준 예시):

```
conda install pytorch==1.9.0 torchvision==0.10.0 torchaudio==0.9.0 -c  
pytorch
```

또는

```
pip install torch==1.9.0 torchvision==0.10.0 torchaudio==0.9.0
```

- 주피터 노트북 설치:

```
pip install jupyter notebook  
jupyter notebook
```

코랩(Colab) 사용하기

- 구글 코랩은 클라우드 기반 무료 주피터 노트북 환경
- 고성능 PC/서버 없이도 대용량 데이터 분석 가능
- 보통 개인 PC 성능이 부족하면 코랩을 많이 사용

2.4 파이토치 코드 맛보기

데이터셋 소개

- 데이터셋 파일: car_evaluation.csv
- 입력 특징(컬럼, X):
 1. price (자동차 가격)
 2. maint (자동차 유지 비용)
 3. doors (자동차 문 개수)
 4. persons (탑승 인원)
 5. lug_capacity (수하물 용량)
 6. safety (안전성)
- 출력(라벨, y): output (차 상태)
 - unacc (허용 불가능), acc (허용 가능), good, vgood

필요한 라이브러리 설치

```
pip install matplotlib seaborn scikit-learn
```

설치 시 충돌 방지를 위해 옵션 추가:

```
pip install --upgrade matplotlib --use-feature=2020-resolver
pip install --upgrade seaborn --use-feature=2020-resolver
pip install --upgrade scikit-learn --use-feature=2020-resolver
```

라이브러리 불러오기

```
import torch
import torch.nn as nn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import seaborn as sns
%matplotlib inline
```

데이터 불러오기 & 확인 (not in colab)

```
dataset = pd.read_csv('../chap02/data/car_evaluation.csv')
print(dataset.head()) # 앞부분
print(dataset.tail()) # 뒷부분
```

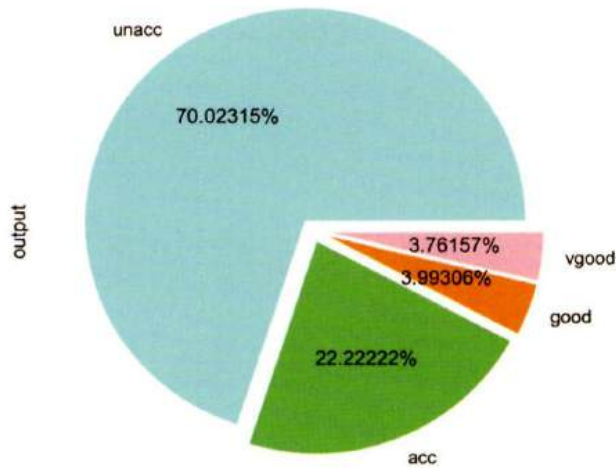
출력 예시:

```
price maint doors persons lug_capacity safety output
0 vhigh vhigh    2    2    small    low unacc
1 vhigh vhigh    2    2    small    med unacc
```

데이터 분포 시각화 (라벨 분포)

```
fig_size = plt.rcParams["figure.figsize"]
fig_size[0], fig_size[1] = 8, 6
plt.rcParams["figure.figsize"] = fig_size

dataset.output.value_counts().plot(
    kind='pie', autopct='%0.05f%%',
    colors=['lightblue', 'lightgreen', 'orange', 'pink'],
    explode=(0.05, 0.05, 0.05, 0.05)
)
```



결과:

- unacc 약 70%
- acc 약 22%
- good, vgood → 매우 적음

즉, 불균형 데이터셋임을 알 수 있음

데이터 전처리 (범주형 → 숫자형)

모든 컬럼이 범주형이므로 category 타입으로 변환 후 숫자로 인코딩

```

categorical_columns = ['price','maint','doors','persons','lug_capacity','safet
y']
for category in categorical_columns:
    dataset[category] = dataset[category].astype('category')

price = dataset['price'].cat.codes.values
maint = dataset['maint'].cat.codes.values
doors = dataset['doors'].cat.codes.values
persons = dataset['persons'].cat.codes.values
lug_capacity = dataset['lug_capacity'].cat.codes.values
safety = dataset['safety'].cat.codes.values
  
```

넘파이 배열로 합치기

```
categorical_data = np.stack(
    [price, maint, doors, persons, lug_capacity, safety], 1
)
print(categorical_data[:10]) # 변환된 값 일부 출력
```

np.stack vs np.concatenate

- np.concatenate : 같은 차원의 배열 연결 (축 유지)
- np.stack : 새로운 축을 추가해 합침 (차원 증가)

np.stack은 두 배열의 차원이 동일해야 함 → 차원이 다르면 ValueError: all input arrays must have the same shape.

텐서로 변환

```
categorical_data = torch.tensor(categorical_data, dtype=torch.int64)
print(categorical_data[:10])
```

- categorical_data (넘파이 배열)을 PyTorch 텐서로 변환

출력 예시:

```
tensor([[3, 3, 0, 0, 2, 1],
        [3, 3, 0, 0, 2, 2],
        [3, 3, 0, 0, 2, 0],
        ...
        [3, 3, 0, 1, 2, 1]])
```

→ 6개의 입력 특징(가격, 유지비, 문 개수, 탑승 인원, 수하물, 안전성)이 정수로 변환된 모습

레이블을 텐서로 변환

```
outputs = pd.get_dummies(dataset.output) # 원-핫 인코딩
outputs = outputs.values
outputs = torch.tensor(outputs).flatten()
```

```
print(categorical_data.shape)
print(outputs.shape)
```

1. pd.get_dummies()

- output (차 상태: unacc, acc, good, vgood)을 원-핫 인코딩으로 변환
- 예: unacc → [1,0,0,0], acc → [0,1,0,0]

2. .values → 넘파이 배열로 변환

3. torch.tensor(outputs) → 텐서 변환

4. .flatten() → 2차원 배열을 1차원으로 펼침

이렇게 하면 입력(categorical_data)과 출력(outputs) 모두 텐서 형태

get_dummies()

```
import pandas as pd
import numpy as np

data = {
    'gender': ['male','female','male'],
    'weight': [72,55,68],
    'nation': ['Japan','Korea','Australia']
}
df = pd.DataFrame(data)
pd.get_dummies(df)
```

출력:

	weight	gender_female	gender_male	nation_Australia	nation_Japan	nation_Korea
0	72	0	1	0	1	0
1	55	1	0	0	0	1
2	68	0	1	1	0	0

→ 문자형 데이터(성별, 국가)가 0/1 숫자로 변환됨. 머신러닝 학습 가능

ravel(), reshape(), flatten()

```
a = np.array([[1,2],
              [3,4]])
print(a.ravel())    # [1 2 3 4]
print(a.reshape(-1)) # [1 2 3 4]
print(a.flatten())  # [1 2 3 4]
```

모두 2차원 배열을 1차원 벡터로 바꾸는 방법.

- `ravel()` : 뷰(view) 반환 (원본 영향 가능)
- `flatten()` : 복사본 반환
- `reshape(-1)` : 가장 많이 쓰이는 방식

범주형 컬럼을 N차원으로 변환 (임베딩 준비)

```
categorical_column_sizes = [len(dataset[column].cat.categories) for column in categorical_columns]

categorical_embedding_sizes = [
    (col_size, min(50, (col_size+1)//2)) for col_size in categorical_column_sizes
]

print(categorical_embedding_sizes)
```

- 각 범주형 컬럼의 고유값 개수(`col_size`)와
- 임베딩 차원(`min(50, (col_size+1)//2)`)을 튜플로 저장

예:

```
[(4,2), (4,2), (4,2), (3,2), (3,2), (3,2)]
```

→ 가격(4종류 → 임베딩 2차원), 유지비(4종류 → 2차원), ... 안전성(3종류 → 2차원)

데이터셋 분리

```
total_records = 1728
test_records = int(total_records * .2) # 전체의 20% = 테스트셋
```

```
categorical_train_data = categorical_data[:total_records - test_records]
categorical_test_data = categorical_data[total_records - test_records:]
```

```
train_outputs = outputs[:total_records - test_records]
test_outputs = outputs[total_records - test_records:]
```

- 전체 1728개 중 80% (1383개) → 훈련용
- 20% (345개) → 테스트용

데이터셋 분리 확인

```
print(len(categorical_train_data))
print(len(train_outputs))
print(len(categorical_test_data))
print(len(test_outputs))
```

출력 결과:

```
1383
1383
345
345
```

전체 1,728개 데이터 중 80% = 1383개는 훈련용, 20% = 345개는 테스트용으로 분리됨을 확인.

모델 네트워크 생성

```
class Model(nn.Module):
    def __init__(self, embedding_size, output_size, layers, p=0.4):
        super().__init__()
        self.all_embeddings = nn.ModuleList([nn.Embedding(ni, nf) for ni, nf in
embedding_size])
        self.embedding_dropout = nn.Dropout(p)

        all_layers = []
```

```

num_categorical_cols = sum((nf for ni, nf in embedding_size))
input_size = num_categorical_cols

for i in layers:
    all_layers.append(nn.Linear(input_size, i))
    all_layers.append(nn.ReLU(inplace=True))
    all_layers.append(nn.BatchNorm1d(i))
    all_layers.append(nn.Dropout(p))
    input_size = i

all_layers.append(nn.Linear(layers[-1], output_size))
self.layers = nn.Sequential(*all_layers)

def forward(self, x_categorical):
    embeddings = []
    for i, e in enumerate(self.all_embeddings):
        embeddings.append(e(x_categorical[:, i]))
    x = torch.cat(embeddings, 1) # 임베딩 합치기
    x = self.embedding_dropout(x) # 드롭아웃
    x = self.layers(x)          # 신경망 통과
    return x

```

설명

- nn.Embedding: 범주형 데이터를 벡터로 변환
- Dropout: 과적합 방지
- Linear: 완전연결층
- ReLU: 활성화 함수
- BatchNorm1d: 정규화로 학습 안정화
- forward: 실제 데이터를 네트워크에 통과시키는 과정

모델 객체 생성

```

model = Model(categorical_embedding_sizes, 4, [200,100,50], p=0.4)
print(model)

```

출력 구조 예시

- 6개 범주형 변수 각각 Embedding(ni, nf)으로 변환
- 은닉층: 200 → 100 → 50
- 출력층: 4 (자동차 상태: unacc, acc, good, vgood)

손실 함수 & 옵티마이저 정의

```
loss_function = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

다중 클래스 분류이므로 CrossEntropyLoss 사용, 옵티마이저는 Adam.

CPU/GPU 지정

```
if torch.cuda.is_available():
    device = torch.device('cuda')
else:
    device = torch.device('cpu')
```

GPU가 있으면 CUDA, 없으면 CPU로 실행

모델 학습

```
epochs = 500
aggregated_losses = []
train_outputs = train_outputs.to(device=device, dtype=torch.int64)

for i in range(epochs):
    i += 1
    y_pred = model(categorical_train_data).to(device)
    single_loss = loss_function(y_pred, train_outputs)
    aggregated_losses.append(single_loss)

    if i % 25 == 1:
        print(f'epoch: {i:3} loss: {single_loss.item():10.8f}')

    optimizer.zero_grad()
```

```
single_loss.backward()
optimizer.step()
```

- 500 epoch 동안 학습, 25 epoch마다 손실 출력
- loss: 예측값과 실제값의 차이를 나타냄.

테스트 데이터셋 평가

```
test_outputs = test_outputs.to(device=device, dtype=torch.int64)
with torch.no_grad():
    y_val = model(categorical_test_data)
    loss = loss_function(y_val, test_outputs)
print(f'Loss: {loss:.8f}')
```

- 학습되지 않은 데이터(테스트셋)로 손실값 확인.
- 과적합 여부 판단 가능.

예측 결과 확인

```
print(y_val[:5])
```

- 5개의 예측 결과 출력 (로그 확률 값)
- 가장 큰 값을 선택해야 클래스 예측 가능

```
y_val = np.argmax(y_val, axis=1)
print(y_val[:5])
```

- 예측된 실제 라벨(0, 1, 2, 3 중 하나).

모델 성능 평가

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

print(confusion_matrix(test_outputs, y_val))
```

```
print(classification_report(test_outputs,y_val))
print(accuracy_score(test_outputs, y_val))
```

출력:

- 정확도 (accuracy): 약 75%
- 정밀도 (precision), 재현율 (recall), F1-score 확인

용어 정리

- True Positive (TP): 실제 1, 예측 1
- True Negative (TN): 실제 0, 예측 0
- False Positive (FP): 실제 0인데 1로 예측 (Type I error)
- False Negative (FN): 실제 1인데 0으로 예측 (Type II error)

평가 지표들

- 정확도 (Accuracy)
- 재현율 (Recall)
- 정밀도 (Precision)
- F1-score = (정밀도와 재현율의 조화 평균)